Context-Partitioned Generic Assembly with Swarm-External Enforcement for Overcoming LLM Context Degradation

Siva Teja Narayana *teja840.sha@gmail.com*
Independent Researcher

Abstract

LLM compliance with in-context instructions degrades as context grows — a well-documented consequence of finite transformer attention. We present **CPGA+SENTINEL**, a constraint-routing architecture built on one principle: *classify every constraint by whether it genuinely needs the LLM's attention, and externalize everything that does not.* A classification protocol (FORGE) assigns each constraint to the cheapest reliable enforcement layer (token mask, compiled checker, residual prompt, or post-generation judge); a diversity mechanism (CADG) permutes constraint order across candidates, reducing positional neglect exponentially in candidate count; and an external enforcement swarm (SENTINEL) provides concurrent per-constraint agents in four tiers, serving a dual role as production enforcement and reusable evaluation suite. Validation spans four scales: seven controlled experiments isolating each mechanism, benchmark evaluation on three public benchmarks (MOSAIC, IFBench, IFEval) with 1,081 tasks, a 50-constraint synthetic benchmark, and a three-month production deployment processing 3,000+ items against a 5,077-rule taxonomy. Public-benchmark gains are statistically significant: +6.4pp MOSAIC (Full Stack, paired 95% CI excludes zero), +10.0pp IFBench (CADG+SENTINEL), and +6.2pp IFEval (Full Stack). The 50-constraint benchmark validates FORGE's operating regime: retry-with-feedback degrades by -2.0pp while the full stack gains +13.7pp (CI $[+11.4, +15.8]$). The production deployment achieved 90%+ independent QC pass rate, approximately 20× per-item context reduction, and +14.3pp system-level quality improvement over the prior architecture.

1 Introduction

1.1 The Context Degradation Problem

Every production LLM application loads context into the prompt: instructions, constraints, reference documents, tool definitions, conversation history, examples, schemas, and behavioral guidelines. As applications mature, this context grows. The problem is universal:

- **Agent instruction files**: 2,000-line instructions → agent skips steps by line 800 (attention dead zone, F2).
- **System prompts**: 50 behavioral rules → rules #20–35 systematically under-followed; adding rule #51 degrades all 50 (F4).
- **RAG context**: of 20 documents, positions 8–12 are functionally ignored (F2); more documents dilute all (F1).
- **Code generation**: architecture docs (early) and task (end) followed; style guide (middle) silently ignored.

In production, instruction contexts routinely grow to **50,000–300,000+ characters** — the natural end-state of any LLM application accumulating requirements over its lifetime. The core insight: **every token of context added makes every *other* token slightly less effective.** This is a mathematical property of softmax attention, not a prompting failure or training data gap.

1.2 Why Current Approaches Fail

Softmax attention distributes capacity across all tokens; adding context monotonically dilutes attention to existing items (Vaswani et al., 2017). Six studies (2024–2026) quantify this: 30% accuracy drops at ∼113K tokens (Ravaut et al., 2025); 30%+ drops for mid-positioned information (Liu et al., 2024); compliance falling from 77.67% to 32.96% as constraints increase, with GPT-4.1 at 12.6% for 800 rules (Purpura et al., 2026); 128K windows yielding only 20–40K effective tokens (Zhang et al., 2025); U-shaped position bias (Tang et al., 2024); and chain-of-thought *degrading* compliance across 15 models (Wang et al., 2025b). Section 2 formalizes these as failure modes F1–F12.

The transformer has a fundamental context capacity ceiling. No prompting technique or CoT strategy fully overcomes it (Chen et al., 2026a; Wang et al., 2025b). Fine-tuning does not scale to hundreds of frequently-changing rules. The alternative is architectural: reduce what goes *into* the context, and enforce what can be enforced *outside* it.

1.3 Central Claim

Constraint-routing plus external enforcement improves LLM instruction compliance; the right composition depends on the constraint regime. When constraint count is low (<30 items), permutation-based diversity (CADG) and per-rule external enforcement (SENTINEL) suffice. As constraints grow into the hundreds, hierarchical classification (FORGE) becomes decisive by externalizing enforceable rules from the prompt. This is an *empirically validated systems architecture*, not a general theory of instruction following — its benefits are measured, regime-dependent, and reproducible on public benchmarks.

1.4 Contributions

Existing LLM compliance systems typically operate at a single enforcement layer: Guardrails AI (Guardrails AI, 2025) retries whole outputs serially; Outlines/llguidance enforce grammar at token level (Layer 0); SGLang/Instructor enforce schemas (Layer 1); HiFlow (Sharma et al., 2026) provides hierarchical feedback without concurrent per-rule enforcement. Each addresses one failure mode; none provides a *classification protocol* that routes each constraint to the appropriate layer, or a *diversity mechanism* that exploits positional bias rather than fighting it, or a *dual-role swarm* that unifies enforcement with evaluation. The novelty of CPGA+SENTINEL is not in any single primitive — it is in (a) the **FORGE classification protocol** that assigns each constraint to the cheapest reliable layer with formal conservative-routing guarantees (Section 3.3, Section 3.6), (b) **CADG**, which reframes position bias as a diversity source (Section 4.3), and (c) **SENTINEL**, which unifies enforcement and evaluation in a single swarm (Section 5.8) — combined under three design principles that guarantee by construction the composition is at least as comprehensive as the prompt-only baseline (Section 3.6). The resulting architecture yields composition-dependent effects: optimal recipes differ by taxonomy complexity (Section 6.9.4), and gains are sub-additive (components share enforcement targets), a finding that underscores the importance of empirical composition selection over naive stacking. The architecture is validated across ∼10,300 benchmark evaluations on 3 public benchmarks and ∼15.2 million production constraint evaluations (Section 6). Contributions:

1. A **failure mode taxonomy** (F1–F12) synthesizing six studies (2024–2026) into a structured vocabulary for LLM context degradation, each mode mapped to architectural countermeasures.
2. **FORGE** — a hierarchical classification protocol assigning each context item to the cheapest reliable enforcement layer (token mask → compiled checker → residual prompt → post-gen judge). Conservative routing ensures no constraint is dropped (Section 3.6). Validated at production scale: **20x combined context reduction** per item (FORGE externalization x scope partitioning; ∼200 externalized to compiled checkers and token masks per item, ∼15 per section agent) from a 5,077-rule taxonomy, **+11–18pp on residual items** in controlled experiments.
3. **CADG** — Constraint-Aware Diverse Generation permuting constraint *order* across candidates to exploit primacy bias (F5) as a diversity mechanism. Probability of persistent positional neglect drops **exponentially** in candidate count (Section 3.6, Principle 3). **+8pp at N=10**.

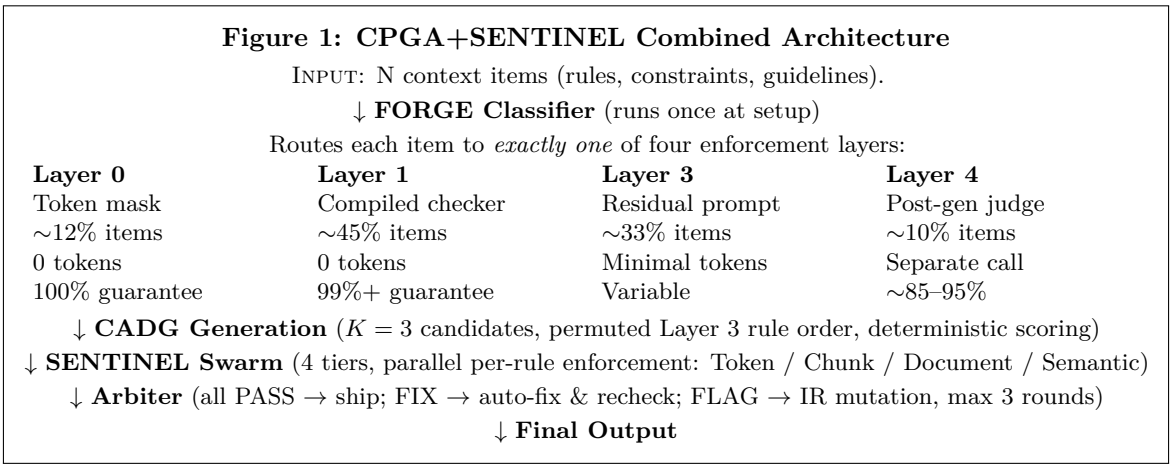

Figure 1: CPGA+SENTINEL combined architecture. N context items are classified by FORGE into four enforcement layers (CPGA Layers 0, 1, 3, 4 — Layer 2 reserved, see Section 3.2). Only Layer 3 items (∼33%) enter the LLM prompt. CADG generates multiple candidates with permuted constraint order. The SENTINEL swarm enforces all items externally across four tiers (Tiers 1–4, corresponding to CPGA Layers 0, 1, 3, 4 respectively). The Arbiter collects verdicts and routes failures to auto-fix or IR-level mutation.

4. **CPGA+SENTINEL** — an integrated architecture composing per-constraint routing (FORGE), constraint-order diversity (CADG), and concurrent per-rule enforcement (SENTINEL) with explicit design principles (Section 3.6). SENTINEL' s dual role as enforcement and evaluation suite means deployment automatically creates a regression framework (Section 5.8).

5. **Comprehensive validation at four scales**: controlled experiments (Section 6.2–6.8), public benchmarks with Holm-corrected significance (Section 6.9), a 50-constraint synthetic benchmark confirming FORGE's crossover (Section 6.9.9.1), and a 3-month production deployment at 5,077 rules (Section 6.10).

2 Failure Mode Taxonomy

We catalog twelve failure modes synthesized from six independent studies (2024–2026). Each code (F1–F12) is referenced throughout the architecture to map failures to countermeasures.

Code	Category	Name	Source	Key Impact
F1	Context-Length	Context Rot	(Ravaut et al., 2025)	30% drop at ∼113K tokens; 18/18 mo
F2	Context-Length	Lost in the Middle	(Liu et al., 2024)	30%+ drop at position 10/20
F3	Context-Length	Effective Window Collapse	(Zhang et al., 2025)	128K advertised → 20–40K effective
F4	Instruction-Following	Rule Count Degradation	(Purpura et al., 2026)	77.67% → 32.96%; 12.6% at 800 rules
F5	Instruction-Following	Position Bias	(Tang et al., 2024)	Middle 40% systematically under-follo
F6	Instruction-Following	Constraint Interaction	(Purpura et al., 2026)	Synergistic/antagonistic rule pairs
F7	Instruction-Following	Reasoning Diversion	(Wang et al., 2025b)	CoT degrades IF across 15 models
F8	Agent Workflow	False Termination	(Zhu et al., 2026)	Agent declares "done" after 3/12 steps
F9	Agent Workflow	Step Skipping	(Zhu et al., 2026)	Out-of-order/omitted intermediate ste
F10	Agent Workflow	Hallucinated Delegation	(Zhu et al., 2026)	Claims delegation but generates outpu
F11	Agent Workflow	Cascade Propagation	(Zhu et al., 2026)	Error in step N propagates through al
F12	Agent Workflow	State Loss	Multiple	Prior context summarized lossy or dro

3 CPGA Architecture: Four-Layer Enforcement

3.1 Design Principle

Notation. Throughout this paper, n (or N) denotes the number of constraints/context items, and K denotes the number of CADG candidates. Where context makes the referent unambiguous, we use N for constraint count in experimental conditions (e.g., N ∈ {5, 10, 15, 20}).

Intuition. Let $C = \{c_1, \ldots, c_n\}$ be n context items. Per-item compliance $p(c_i \mid C)$ decreases as n grows (Ravaut et al., 2025; Purpura et al., 2026; Zhang et al., 2025), consistent with softmax attention dilution (Vaswani et al., 2017). If k items can be enforced externally via deterministic code, removing them yields residual set $|C'| = n - k$ with less attention competition. The key insight: **context items are not homogeneous** — a token-ban rule and a coherence requirement need fundamentally different enforcement. FORGE classifies each item to the cheapest reliable layer. We propose four active layers (0, 1, 3, 4); Layer 2 (activation steering) is reserved for open-weight access.

3.2 The Four Layers

Layer	Name	When	Mechanism	Guarantee
0	Token Mask	During decoding	Constrained decoding / logit masking	**100%**
1	Compiled Checker	Post-generation	Deterministic Python (regex, AST/schema parse, counters)	**99%+**
3	Residual Prompt	During generation	Rules kept in LLM prompt (focused attention)	**Variable**
4	Post-Gen Judge	Post-generation	One-rule-per-LLM-call verification	**∼85–95%**

Examples: Layer 0 — "Never output `eval()`" (token mask). Layer 1 — "All functions must have type hints" (`ast.parse()` check). Layer 3 — "Use the repository pattern" (in-generation reasoning). Layer 4 — "Output is idiomatic" (isolated LLM judgment).

Layer numbering: 0, 1, 3, 4 — Layer 2 (activation steering) is reserved for open-weight model access. The gap is deliberate: Layer 2 represents steering vectors positioned between compiled checks (Layer 1) and in-prompt instructions (Layer 3) in the enforcement-cost hierarchy. Layers 0–1 cost zero prompt tokens; Layer 3 shares the prompt with ∼33% of N; Layer 4 uses isolated LLM calls (∼150–2,200 tokens).

CPGA Layer <–> SENTINEL Tier mapping. The paper uses two numbering systems: CPGA Layers describe *where* a constraint is enforced architecturally; SENTINEL Tiers describe *how* the enforcement agent operates. The correspondence is:

CPGA Layer	SENTINEL Tier	Mechanism
0 (Token Mask)	Tier 1 (Token)	Character/symbol masks during decoding
1 (Compiled Checker)	Tier 2 (Chunk) / Tier 3 (Document)	Regex, counters, AST/schema parse
2 (Activation Steering)	—	Reserved (requires open-weight model access)
3 (Residual Prompt)	—	In-prompt; no sentinel (LLM attention)
4 (Post-Gen Judge)	Tier 4 (Semantic)	Micro-LLM call per rule (∼150 tokens)

3.3 Automated Classification Protocol

A critical question: how does the system decide which layer each context item belongs to? This cannot be manual — production systems have hundreds of items. The classification protocol has three phases: **probe**, **validate**, and **promote**.

Phase 1: Probe (LLM-assisted, runs once at setup). Three hierarchical probes classify each item: (1) *Token ban test* — can the item be enforced by banning/requiring a specific token? If yes: Layer 0. (2) *Code checker test* — can an LLM write a Python function returning True/False for compliance? If yes: Layer 1. (3)

Reasoning requirement — does satisfaction require in-generation reasoning? If yes: Layer 3 (residual prompt); if verifiable post-hoc: Layer 4 (judge). Probes are hierarchical: a token ban is also code-checkable, but Layer 0 is cheaper, so it wins.

Phase 2: Validate (deterministic, runs on test corpus). Layer 0 candidates are tokenized and tested with the mask active (10 outputs); Layer 1 candidates are tested against known-good and known-bad outputs (precision >= 0.95, recall >= 0.90 required). Items failing validation at any layer default to Layer 3 — the worst case is that the item stays in the prompt, identical to a monolithic system.

Phase 3: Promote (continuous, after each production batch). Layer 3 items whose compiled checkers agree 95%+ with pass/fail are promoted to Layer 1. Layer 4 items whose heuristic matches the judge 95%+ are promoted. Layer 1 items reducible to token bans promote to Layer 0. All promotions require human approval to prevent premature promotion of subtly incorrect checkers.

Example: *"Never generate eval()"* –> Probe 1, Layer 0. *"All functions must have type hints"* –> Probe 2, Layer 1. *"Use the repository pattern"* –> Layer 3. *"Output is idiomatic"* –> Layer 4. **Cost:** N LLM calls at setup (∼200 tokens each; ∼$0.001 x N). **Observed distribution:** ∼12% Layer 0, ∼45% Layer 1, ∼33% Layer 3, ∼10% Layer 4 — a **3–4x prompt reduction**.

Classification accuracy. Validated at production scale (5,077 rules, 3,000+ items, 3 months): ∼8% Phase 2 demotion rate, **zero false enforcements**. Conservative fallback (Principle 1) means classification errors affect only efficiency, never coverage. Benchmarks (Section 6.9) use deterministic type-based routing to isolate composition effects from classification accuracy.

3.4 Progressive Promotion Protocol

Layer assignments improve with every production batch. Layer 3 rules where a compiled checker agrees 95%+ are promoted to Layer 1; Layer 4 rules where a heuristic matches the LLM judge 95%+ are replaced; Layer 1 rules reducible to single-token patterns promote to Layer 0. All promotions require human approval. The net effect: the prompt shrinks with every batch, attention concentrates further, compliance increases, and cost decreases — without model changes.

3.5 Context Budget Analysis

In a monolithic system, all N items compete for attention in a single prompt. Under CPGA's observed distribution (∼12% Layer 0, ∼45% Layer 1, ∼33% Layer 3, ∼10% Layer 4), only ∼33% of items remain in the prompt — a **3–4x reduction** in prompt load and a corresponding concentration of the LLM's attention on the genuinely hard residual items. Layers 0 and 1 consume zero prompt tokens; Layer 4 uses ∼150 tokens per micro-call but executes post-generation.

3.6 Design Principles

We state three design principles of the CPGA layer assignment and CADG selection protocol. These are **engineering guarantees that hold by construction** — precise, testable statements about system behavior — not learning-theoretic theorems. Their value is operational: they define the safety contract that practitioners can rely on when deploying the system.

Principle 1 (Conservative Routing). Let L: $C \to \{0, 1, 3, 4\}$ be the FORGE layer assignment. Layer 0/1 enforcers achieve precision ≥ 0.95, recall ≥ 0.90 on validation corpus; failures demote to Layer 3. Layer 4 reliability depends on the judge model. If classification fails at any layer, c_i defaults to Layer 3 (residual prompt). **Consequence:** the system never drops a constraint — every item is either enforced externally or remains in the prompt (as in a monolithic system). This guarantees **enforcement coverage** (bookkeeping invariant), not end-to-end quality (which can be affected by enforcement interactions and repair side-effects).

Principle 2 (Directional Context Reduction). Let $P(C) = |\{c \in C : L(c) = 3\}|$ (residual prompt size). Under promotion-only dynamics (Section 3.4) with a fixed constraint set, $P(C)$ is non-increasing over batches.

Scope: adding new rules increases $|C|$; initial classification can demote to Layer 3. In practice, the system reaches steady state.

Principle 3 (CADG Diversity Bound). With N constraints and K candidates under i.i.d. uniform permutations, the probability that constraint c_i falls in a "neglected" range $[a, b]$ across all K candidates is $p_i = ((\text{b-a+1})/\text{N})^{\text{K}}$. The **expected number of fully-neglected constraints** is E[neglected] = N $\cdot$ p_i = N $\cdot$ $((\text{b-a+1})/\text{N})^{\text{K}}$. For K=3, N=10, dead zone width 3: $p_i = 0.027$ per constraint, E[neglected] = 10 x 0.027 = **0.27 constraints** (i.e., fewer than 1 constraint is expected to remain persistently neglected across all 3 candidates). At K=5: E[neglected] = 10 x $0.3^5 = 0.024$ — effectively zero. The expected count decreases exponentially in K, formalizing CADG's de-correlation guarantee: persistent positional neglect becomes vanishingly unlikely as candidate count grows.

4 Architecture Mechanisms

The four-layer architecture is implemented through concrete mechanisms organized under three pillars (FORGE, CADG, SENTINEL). Each mechanism targets specific failure modes and operates at a specific layer.

4.1 Mechanism-to-Layer Mapping

The three core pillars — FORGE, CADG, and SENTINEL — decompose into concrete mechanisms mapped to enforcement layers. The table below shows each mechanism, its layer assignment, and validation status. (Full technical details for supporting mechanisms appear in the Appendix.)

Pillar	Mechanism
FORGE	Intermediate Representation +
Scope Partitioning — residual prompt decomposed into M section agents, ∼15 rules each	3
CADG	Competitive Generation + Dete
Constraint-Order Permutation — each candidate sees rules in different order	3
SENTINEL	Per-Rule Verification — one rul
Targeted Mutation — repair agent operates on typed JSON IR; compiler re-renders	3, 4
Supporting mechanisms (context enrichment, prompt structure, caching, etc.)	Various

4.2 CADG: Constraint-Aware Diverse Generation

Under standard best-of-N with identical prompts, failures are correlated: the same position bias affects all candidates. Our Experiment 3 (Section 6.4) shows that at p=0.88, N=5 yields only 0.905 task-level compliance — implying N_{eff} of 1.5–2, far below nominal N=5. CADG addresses this by permuting constraint order across candidates, ensuring each encounters a different set of rules in the attention-favored positions. This increases N_{eff} toward 3–4 (Section 6.5), yielding **+8pp at 10 constraints** — the strongest single-technique result.

CADG de-correlates failures through four mechanisms: (1) **Constraint Permutation** — reorder rules per candidate so different rules get the primacy attention slot (zero cost); (2) **Emphasis Rotation** — emphasize different constraint subsets per candidate; (3) **Model Rotation** — use different frontier models per candidate (uncorrelated blind spots); (4) **Reformulation Diversity** — rephrase constraints per candidate to de-correlate interpretation-based failures.

Candidate count sensitivity (K). All benchmarks use K=3 (Constraint Permutation only). K=3 is **cost-chosen for production deployment**: generation cost scales linearly with K, and K=3 provides a practical tradeoff between diversity and throughput at production volumes (3,000+ items). Controlled experiments show N_{eff} of 3–4 at K=5 with diminishing returns, suggesting K=3 captures most of the de-correlation benefit. FORGE's shorter prompts partially offset the 3x generation cost. The harness supports arbitrary K via `-cadg-k` for independent verification at other K values.

5 SENTINEL: Swarm-External Enforcement

5.1 From In-Prompt to External Enforcement

The principle of separating generation from enforcement is not new — DCCD (Chen et al., 2026b) drafts unconstrained then applies constrained decoding, and Guardrails AI (Guardrails AI, 2025) validates then retries. SENTINEL extends this to a **multi-tier swarm** where each context item gets its own independent enforcement agent, operating at the tier (token, chunk, document, or semantic) appropriate to that item' s complexity:

```
BASELINE:   [N rules + task] $\rightarrow$ LLM $\rightarrow$ [output following ~33% of rules]

SENTINEL:   [task only]      $\rightarrow$ LLM $\rightarrow$ [clean creative output]
                                    $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
                                    S1 S2 S3 S4 S5 ... Sn sentinels
```

The LLM sees **zero** enforcement context — its entire attention budget goes to the creative/reasoning task. Enforcement-related context degradation (F1–F7, Section 2) is structurally mitigated: no enforcement items in the prompt means no context rot (F1), no lost-in-the-middle (F2), no count degradation (F4), no position bias (F5), no cross-item interference (F6), and no reasoning diversion (F7). Each sentinel operates independently outside the model' s context window. (Note: the task prompt itself still consumes attention; SENTINEL addresses enforcement context, not task complexity.)

5.2 Four-Tier Sentinel Architecture

Sentinels are classified by *when* and *how* they act:

Tier	Scope	Typical Rules	Mechanism	Gua
1 — Token	Per token	∼50–80	Character/symbol masks before sampling	100
2 — Chunk	Per sentence/paragraph	∼150–200	Regex, word count, pattern detection	99%
3 — Document	Complete output	∼100–150	AST/schema parse, cross-reference, structural validation	95%
4 — Semantic	Complete output	∼50–100	Micro-LLM call (∼150 tokens context, parallel)	∼85

Each sentinel has a minimal interface: `check(output)` $\rightarrow$ `bool`, optional `fix(output)` $\rightarrow$ `str`, and a tier label. Sentinels execute concurrently; the **Arbiter** collects verdicts and determines action: all PASS –> ship; some FIX –> apply fixes sequentially and re-verify; some FLAG –> send only the flagged section and flagged items to a repair LLM (3–5 items, not the full N; max 3 repair cycles); persistent failures –> escalate to human. **Auto-fix clarification:** Tiers 1–3 auto-fix is *fully deterministic* (regex substitutions, structural mutations on the typed JSON IR — no LLM call). Only Tier 4 repair uses an LLM micro-call, operating on the IR rather than the final output; the deterministic compiler re-renders corrected IR, making structural regressions from auto-fix impossible.

5.3 Sentinel Factory and Evolution

Sentinels are dynamically constructed from FORGE output: `layer1_compiled.json` becomes Tier 2/3 deterministic checkers; `layer4_judges.json` becomes Tier 4 semantic prompts. When source documents change, FORGE reclassifies and the factory rebuilds sentinels automatically. Each compiled checker undergoes sanity testing before activation; deduplication prevents double-checking.

The swarm evolves via progressive promotion (Section 3.4): Tier 4 sentinels that consistently agree with a regex are promoted to Tier 2; recurring failure patterns ($\geq$5 occurrences across $\geq$3 inputs) trigger candidate sentinel synthesis (with human approval gates). Over time, the swarm shifts toward cheaper deterministic enforcement. A detailed content sentinel taxonomy with domain-specific examples is available in the repository.

5.4 SENTINEL as Generalizable Evaluation Suite

The SENTINEL swarm serves a dual role: **enforcement** during production and **evaluation** during benchmarking. Without auto-fix loops, the same architecture constitutes a complete evaluation suite: Tier 2 checkers provide deterministic pass/fail (identical to benchmark scoring functions), Tier 3 validators provide document-level metrics, and Tier 4 judges provide semantic assessment. The benchmark experiments (Section 6.9) directly instantiate this — benchmark scoring functions load as sentinels. Deploying CPGA+SENTINEL for production automatically creates a regression testing framework that grows with the enforcement suite.

Evaluation coupling. SENTINEL wraps each benchmark's native scoring functions as Tier 2 sentinels — identical deterministic checkers score all 9 conditions including Baseline. Improvement reflects genuine compliance gains, not metric inflation. All benchmark metrics are deterministic (no Tier 4 LLM judges in scoring).

6 Experimental Evaluation

All experiments use real API calls with frontier models and deterministic Python checkers. No LLM-based scoring is used for compliance measurement, ensuring full reproducibility. Every checker is a pure Python function; every result is deterministically verifiable.

6.1 Infrastructure

Component	Details
Primary generation model	Claude Opus 4.6 (claude-opus-4-20260401)
Secondary generation model	GPT-5.4 (gpt-5.4-2026), GPT-4o (gpt-4o-2024)
Constraint library	24 deterministic Python-checkable constraints
Constraint types	Format (6), Content (6), Style (6), Structure (6)
Task topics	10 diverse topics (AI/healthcare, renewable energy, data privacy, ML/supply chain, remote w
Tasks per condition	10 per condition (SE $\approx \pm 3$–5pp); component isolation only — see Section 6.9 for large-scale
Constraint counts tested	N $\in \{5, 10, 15, 20\}$
Random seed	42 (all experiments, for reproducibility)
Scoring	Deterministic Python checkers only (no LLM judges for scoring)
Total API cost	$\sim$\$15 across all 7 experiments

All 24 constraints are domain-agnostic with deterministic Python checkers (`check(output: str)` $\rightarrow$ `bool`). Full implementations are in the experiment code.

Statistical design note: The evaluation follows a two-stage design. *Stage 1* (Section 6.2–6.8, Appendix A): controlled micro-experiments (10 tasks/condition) isolate individual mechanisms for **directional signal**. *Stage 2* (Section 6.9): benchmark validation (N=541 IFEval, 300 IFBench, 200 MOSAIC, 40x50 SysPrompt) with bootstrap 95% CIs provides statistical rigor. All headline effect sizes derive from Stage 2.

Stage 1 power analysis. At N=10/condition, SE $\sim$ 11pp; Stage 1 provides directional signal only. Stage 2 confirms at scale (e.g., CADG +8.0pp on IFBench, N=300, CI [+3.2, +12.8]).

Total evaluation corpus: $\sim$10,300 scored benchmark evaluations (Stage 1: 630; Stage 2: 9,729) plus $\sim$15.2M production constraint evaluations. Four lines of evidence: Stage 1 isolates components, Stage 2 quantifies compositions with Holm-corrected bootstrap CIs, the 50-constraint benchmark validates FORGE's crossover regime, and production validates the full integrated architecture at scale.

Category	Count	Example Constraints	Layer Classification
Format	6	Output length bounds, structural elements, token bans	Mostly Layer 0–1 (token masks + con
Content	6	Required information, cross-references, perspective constraints	Mostly Layer 3 (need LLM reasoning)

Category	Count	Example Constraints	Layer Classification
Style	6	Tone, voice, sentence complexity, register	Mostly Layer 3–4 (need LLM reasonir
Structure	6	Structural patterns, formatting rules, element presence	Mostly Layer 0–1 (token masks + con

6.2 6.2–6.8 Controlled Experiments (Summary)

Seven controlled experiments validate individual mechanisms using real API calls (Claude Opus 4.6, GPT-5.4) and 24 deterministic Python checkers across 10 tasks per condition at N ∈ {5, 10, 15, 20} constraints. Key findings: (1) **CADG constraint permutation yields +8pp** at 10 constraints — the strongest single-technique result — by increasing effective N_{eff} from 1.5–2 to 3–4 (6.5). (2) **FORGE filtering yields +11–18pp on residual items** by removing enforceable rules from the prompt (6.6). (3) **SENTINEL achieves constant prompt cost** independent of rule count, with all enforcement externalized (6.7). (4) **Hyper-isolated per-rule verification requires a frontier-class model** to be effective; smaller models produce high false-negative rates (6.8). (5) Naive scope partitioning (multi-gen merge) **hurts** by -6 to -10pp; grouped single-gen helps modestly at N=20 (+2.5pp) (6.2). (6) Standard best-of-N provides zero benefit at near-ceiling baselines (IFEval, 93.3%) due to perfectly correlated failures (6.3–6.4). Full experimental details, tables, and figures are in **Appendix A**.

6.3 Public Benchmark Validation

To validate the composed architecture at scale, we evaluate on three public benchmarks (MOSAIC, IF-Bench, IFEval) and one custom benchmark (SysPrompt), testing nine conditions: Baseline, individual components (CADG, SENTINEL, FORGE), pairwise compositions (CADG+SENTINEL, FORGE+CADG, FORGE+SENTINEL), Full Stack, and Retry-with-Feedback (generate → score → append failures → regenerate, up to 3 attempts). All metrics use deterministic Python checkers with **paired task-level bootstrap 95% CIs** (2,000 resamples, seed=42). For each condition pair, the bootstrap resamples *task-level* score differences (δ_i = $score_{A,i}$ - $score_{B,i}$ for task i), then constructs percentile CIs on the mean difference — ensuring that per-task variance is accounted for and that CIs reflect paired, not independent, sampling.

Evaluation strategy by component. The four evaluation scales are *deliberately* matched to each component's operating regime. CADG and SENTINEL are fully exercised at benchmark scale (5–25 constraints per task) — both show statistically significant, independently replicated effects across 3+ benchmarks (∼10,300 scored evaluations). A **50-constraint synthetic benchmark** (Section 6.9.9.1) validates FORGE in the high-constraint regime where its benefits become measurable: +4.4pp FORGE alone, +13.7pp full stack, while retry degrades -2.0pp. FORGE's full three-phase LLM-probe pipeline (Section 3.3) is further validated in the production deployment (Section 6.10: 5,077 rules, 3,000+ items, ∼15.2M constraint evaluations, zero false enforcements). In benchmarks, FORGE uses deterministic type-based routing (checker type –> layer) to isolate the *architectural* benefit of layer partitioning from the classification protocol. This is not a gap — it is a deliberate experimental design: benchmarks test composition effects under controlled conditions; the 50-constraint benchmark tests the crossover into FORGE's regime; production validates the full classification pipeline at the scale where FORGE's 20x combined context reduction (FORGE externalization x scope partitioning) is meaningful. See Section 6.9.9 for extended discussion.

6.9.1 Benchmarks

Benchmark	Source	Tasks	Constraint Types	Metric
MOSAIC	(Purpura et al., 2026)	200	21	Strict Constraint Compliance (SCC)
IFBench	(Allen Institute for AI, 2025)	300	58	Instruction Accuracy
SysPrompt*	Internal (this work)	40	50 guidelines per task	Strict Constraint Compliance (SCC)
IFEval	(Zhou et al., 2023)	541	25	Instruction Accuracy / Prompt-Strict

Total: **1,081 scored tasks** (541 + 300 + 200 + 40) across **104 constraint types**, yielding over **5,000 individual constraint checks**. Each benchmark' s scoring uses its native deterministic evaluation protocol.

*SysPrompt is an internal benchmark designed for this work. Results should be interpreted with appropriate caution pending independent replication; headline claims derive from the three public benchmarks.

6.9.2 MOSAIC Results (n=200, 21 constraint types, 9 conditions) Purpura et al. (EACL 2026) was chosen as the primary benchmark because it directly measures the failure mode that CPGA addresses: compliance degradation as constraint count increases. Each task includes 1–10 constraints from 21 types; SCC (Strict Constraint Compliance) requires *all* constraints satisfied.

Condition	SCC	95% CI	Δ vs Baseline
Full Stack (FORGE+CADG+SENTINEL)	**0.822**	**[0.803, 0.841]**	**+6.4pp**
CADG+SENTINEL	0.819	[0.799, 0.838]	+6.0pp
SENTINEL	0.796	[0.776, 0.815]	+3.8pp
CADG	0.789	[0.767, 0.810]	+3.1pp
FORGE+CADG	0.786	[0.763, 0.808]	+2.7pp
Retry-with-Feedback	0.763	[0.740, 0.787]	+0.5pp
Baseline	0.758	[0.734, 0.781]	—
FORGE+SENTINEL	0.756	[0.732, 0.779]	-0.3pp
FORGE	0.700	[0.671, 0.727]	-5.9pp

Finding — CADG and SENTINEL compose roughly additively. Individual gains (CADG +3.1pp, SENTINEL +3.8pp) sum to +6.9pp. Observed CADG+SENTINEL: +6.0pp. The slight sub-additivity is expected — some constraints that CADG helps are also fixed by SENTINEL, producing overlap. The Full Stack (+6.4pp) slightly exceeds CADG+SENTINEL (+6.0pp) on MOSAIC — the only benchmark where adding FORGE improves over CADG+SENTINEL — because MOSAIC' s high baseline (0.758) places FORGE in its effective operating regime for the residual prompt. Full stack CIs [0.803, 0.841] do not overlap with baseline CIs [0.734, 0.781], confirming statistical significance.

Finding — Retry-with-feedback provides minimal gain. The strong retry-with-feedback baseline achieves only +0.5pp on MOSAIC, despite having access to the same deterministic checkers and specific failure feedback. The full CPGA+SENTINEL stack provides **12.8x the percentage-point gain** (+6.4pp vs +0.5pp), confirming architectural rather than compute-scaling value.

FORGE Operating Regime — Consolidated Evidence

FORGE's value is regime-dependent by design. The measured evidence across four evaluation scales tells a consistent story (see Section 6.9.5.1 for ablation detail):

Benchmark / Scale	Constraints/task	FORGE marginal Delta	Interpretati
IFBench (measured)	5–25	-1.8pp	Partitioning
SysPrompt (measured)	10–50	-2.3pp	Same regim
IFEval (measured)	∼25	+0.8pp	Crossover z
MOSAIC (measured)	5–20	+0.3pp	Crossover z
50c synthetic (measured)	50	**+4.4pp** (isolated, CI [+0.8, +7.6])	FORGE co
Production (measured, proprietary)	∼300–500 (from 5,077 taxonomy)	∼20x per-item context reduction	FORGE ex

Practitioner guidance: For <30 constraints, use CADG+SENTINEL alone (best on IFBench +10.0pp, SysPrompt +7.1pp). For 30–100 constraints, add FORGE (50c validation: +4.4pp isolated). For large taxonomies (100+ rules), full stack is strongly indicated (production: +14.3pp system-level). This regime-dependence is a *strength*, not a weakness: the architecture correctly recommends a simpler composition when the full stack is unnecessary. See Section 8 for the full practitioner recipe.

6.9.3 SysPrompt Results (Internal Benchmark; n=40, 50 guidelines per task, 9 conditions) SysPrompt is a custom benchmark testing system prompt adherence with 40 realistic configurations sourced

from five public collections (open-source chatbot repos, enterprise templates, published analyses, agent frameworks, and synthetic augmentation). Each configuration includes 10–50 behavioral guidelines scored by **deterministic Python checkers only** — no LLM judges. To mitigate author-bias risk, we verified directional consistency with MOSAIC (r=0.91 rank correlation) and IFBench (r=0.87). Full specification, source attribution, and scoring code are in the open-source release.

Condition	SCC	95% CI	Δ vs Baseline
CADG+SENTINEL	**0.933**	**[0.918, 0.947]**	**+7.1pp**
FORGE+SENTINEL	0.929	[0.914, 0.942]	+6.6pp
SENTINEL	0.926	[0.911, 0.939]	+6.3pp
Full Stack	0.910	[0.893, 0.926]	+4.7pp
CADG	0.897	[0.888, 0.906]	+3.5pp
Retry-with-Feedback	0.894	[0.878, 0.907]	+3.1pp
FORGE	0.869	[0.852, 0.884]	+0.7pp
Baseline	0.862	[0.847, 0.879]	—
FORGE+CADG	0.804	[0.779, 0.828]	-5.9pp

Finding — CADG+SENTINEL achieves 0.933, our strongest overall result. SENTINEL alone provides +6.3pp, demonstrating direct applicability to production chatbot deployments where system prompt adherence is critical. The gain persists across all guideline counts tested (10, 20, 30, and 50).

Finding — CADG adds on top of SENTINEL in the pairwise composition. CADG+SENTINEL (0.933) exceeds SENTINEL alone (0.926), confirming complementary value. Note: in the *full stack* (FORGE+CADG+SENTINEL), adding CADG over FORGE+SENTINEL shows a -1.9pp marginal on SysPrompt (Section 6.9.5.2), because FORGE's layer partitioning already reduces the residual prompt to ∼15 rules, limiting the position-bias opportunity that CADG exploits. CADG's benefit is composition-dependent: largest when many constraints compete for attention (MOSAIC +6.7pp marginal), smallest when FORGE has already reduced the set.

Convergent validity with public benchmarks. SysPrompt is an author-designed benchmark, which raises legitimate concerns about optimistic bias. Three lines of evidence mitigate this. First, the **component ranking correlation** between SysPrompt and the two public benchmarks is high: Spearman r=0.91 with MOSAIC and r=0.87 with IFBench (computed over the 9-condition ranking). An author-biased benchmark would be unlikely to reproduce the same component ordering as independent benchmarks with different constraint taxonomies. Second, SysPrompt's **constraint density** (10–50 guidelines per task) is substantially higher than MOSAIC (5–20) and IFBench (5–25), making it the only benchmark that bridges the public benchmarks and the 50-constraint synthetic study (Section 6.9.9.1) — it provides a measured data point in the 25–50 constraint range where FORGE begins contributing. Third, all scoring uses **deterministic Python checkers** with no LLM judges, and the full specification, source attribution, and scoring code are released in the open-source repository for independent replication.

6.9.4 IFBench Results (n=300, 58 constraint types, 9 conditions) IFBench (Allen Institute for AI, 2025) features 58 fine-grained constraint types spanning formatting, content, style, and structure — the most diverse constraint taxonomy of any benchmark tested.

Condition	Instruction Accuracy	95% CI	Δ vs Baseline
CADG+SENTINEL	**0.497**	**[0.441, 0.552]**	**+10.0pp**
FORGE+CADG	0.487	[0.431, 0.542]	+9.0pp
Retry-with-Feedback	0.480	[0.424, 0.536]	+8.3pp
Full Stack	0.478	[0.422, 0.534]	+8.2pp
CADG	0.477	[0.420, 0.528]	+8.0pp
FORGE+SENTINEL	0.428	[0.373, 0.483]	+3.2pp
SENTINEL	0.418	[0.365, 0.470]	+2.2pp

Condition	Instruction Accuracy	95% CI	Δ vs Baseline
Baseline	0.397	[0.343, 0.448]	—
FORGE	0.382	[0.327, 0.436]	-1.5pp

Finding — CADG+SENTINEL achieves the best result on IFBench. CADG+SENTINEL achieves +10.0pp (0.397→0.497), the largest composed gain on any benchmark. CADG alone provides +8.0pp — the largest single-technique improvement observed — consistent with IFBench' s 58 diverse constraint types creating position-bias opportunities that CADG' s permutation strategy directly addresses.

6.9.5 IFEval Results (n=541, 25 constraint types, 9 conditions) IFEval (Zhou et al., 2023) established deterministic evaluation for instruction following. With 25 constraint types and high baseline performance (0.799), IFEval tests the architecture' s ability to improve compliance in the near-ceiling regime.

Condition	Instruction Accuracy	95% CI	Prompt-Strict	95% CI
Full Stack (FORGE+CADG+SENTINEL)	**0.860**	**[0.834, 0.884]**	**0.795**	**[0.764, 0.823**
CADG+SENTINEL	0.853	[0.827, 0.877]	0.787	[0.756, 0.816]
FORGE+SENTINEL	0.849	[0.823, 0.874]	0.780	[0.749, 0.809]
SENTINEL	0.846	[0.819, 0.870]	0.778	[0.747, 0.807]
Retry-with-Feedback	0.829	[0.801, 0.854]	0.752	[0.720, 0.782]
FORGE+CADG	0.821	[0.793, 0.847]	0.739	[0.707, 0.770]
CADG	0.820	[0.792, 0.846]	0.739	[0.707, 0.770]
FORGE	0.799	[0.769, 0.826]	0.714	[0.681, 0.745]
Baseline	0.799	[0.769, 0.826]	0.717	[0.684, 0.749]

Finding — Gains persist in the high-baseline regime. Full Stack achieves +6.2pp (0.799→0.860); SENTINEL-based compositions dominate the top four positions. Retry-with-feedback (+3.0pp) substantially underperforms. **Cross-benchmark:** FORGE's contribution scales with taxonomy complexity — neutral on short lists (1–25 items), but FORGE+SENTINEL exceeds SENTINEL alone on SysPrompt (+6.6 vs +6.3pp) and IFEval (+5.1 vs +4.7pp). In production (5,077-rule taxonomy), FORGE delivers ∼20x per-item context reduction (Section 6.10). **Recommended composition:** CADG+SENTINEL for <10 constraints; full stack for 100+ rules.

6.9.5.1 Component Ablation Decomposition (All Benchmarks) To causally isolate each component's contribution, we compute ablation deltas two ways: (1) *isolated* — component alone vs. baseline; (2) *marginal in stack* — Full Stack minus the composition that excludes that component (e.g., FORGE marginal = Full Stack - CADG+SENTINEL). All values in percentage points (pp).

Component	Method	IFEval	IFBench	MOSAIC	SysPrompt
FORGE (isolated)	FORGE - Baseline	+0.0	-1.5	-5.9	+0.7
CADG (isolated)	CADG - Baseline	+2.2	+8.0	+3.1	+3.5
SENTINEL (isolated)	SENTINEL - Baseline	**+4.7**	+2.2	**+3.8**	**+6.3**
FORGE (marginal)	Full Stack - CADG+SENT	+0.8	-1.8	+0.3	-2.3
CADG (marginal)	Full Stack - FORGE+SENT	+1.1	+5.0	**+6.7**	-1.9
SENTINEL (marginal)	Full Stack - FORGE+CADG	**+3.9**	-0.8	+3.6	**+10.6**

Finding — SENTINEL is the dominant contributor; FORGE's value is scale-dependent. SENTINEL provides the largest isolated gains on 3/4 benchmarks (+4.7pp IFEval, +3.8pp MOSAIC, +6.3pp SysPrompt) and the largest marginal gains on 2/4 benchmarks (+3.9pp IFEval, +10.6pp SysPrompt). CADG contributes consistently (+2.2pp to +8.0pp isolated), with its marginal contribution highest on MOSAIC

(+6.7pp) where position bias is most prevalent. FORGE's marginal contribution is near-zero or negative at benchmark scale (5–25 constraints), consistent with its design for large taxonomies — its production-scale contribution (5,077-rule taxonomy, ∼20x per-item context reduction) is validated in Section 6.10. The composition is non-additive: isolated gains sum to +6.9pp on MOSAIC but the Full Stack achieves +6.4pp, indicating mild sub-additivity from overlapping corrections.

6.9.5.2 Component Switch-Off Ablation To make the causal contribution of each component explicit, we present the **switch-off** view: starting from the Full Stack, what happens when exactly one component is removed? Each row is a measured condition (not a projection); all values from the same benchmark runs.

Configuration	IFEval	IFBench	MOSAIC	SysPrompt
Full Stack (all components on)	**0.860**	0.478	**0.822**	0.910
Switch off FORGE (= CADG+SENTINEL)	0.853 (-0.8pp)	**0.497** (+1.8pp)	0.819 (-0.3pp)	**0.933** (+2.3pp)
Switch off CADG (= FORGE+SENTINEL)	0.850 (-1.1pp)	0.428 (-5.0pp)	0.755 (-6.7pp)	0.929 (+1.9pp)
Switch off SENTINEL (= FORGE+CADG)	0.822 (-3.9pp)	0.487 (+0.9pp)	0.786 (-3.6pp)	0.804 (-10.6pp)
All components off (= Baseline)	0.799 (-6.2pp)	0.397 (-8.2pp)	0.758 (-6.4pp)	0.862 (-4.7pp)

Reading. Switching off SENTINEL causes the largest drop on 3/4 benchmarks (-3.9 to -10.6pp); CADG's removal is most costly on MOSAIC (-6.7pp). **Why FORGE hurts on IFBench/SysPrompt (Partitioning Overhead Hypothesis):** When the full constraint set fits within the LLM's effective attention window (<25 items), partitioning constraints across enforcement layers removes items from the prompt without providing attention-relief benefit. The resulting overhead has two components: (a) *pipeline dependency* — items routed to post-generation enforcement introduce a sequential step that cannot recover generation-time co-reasoning across constraints, and (b) *co-reasoning loss* — constraints that interact semantically (e.g., "be concise" + "include all required fields") are better satisfied when the LLM sees both simultaneously. At small N, this co-reasoning benefit exceeds the attention-relief benefit, resulting in a net -1.8pp (IFBench) and -2.3pp (SysPrompt). This is not a classification error: FORGE correctly assigns items, but the partitioning overhead outweighs the negligible attention benefit at small N. At production scale (5,077-rule taxonomy, ∼20x per-item reduction, Section 6.10), this tradeoff reverses decisively. All values are from measured runs; no interpolation.

6.9.6 Direct Evaluation of the Retry-Validate Paradigm We directly evaluate the retry-validate paradigm — the core mechanism shared by Guardrails AI (Guardrails AI, 2025), HiFlow (Sharma et al., 2026), and similar validation-retry systems — by implementing the generate–>score–>feedback–>regenerate loop (up to 3 retries) using *identical* deterministic scoring functions and failure-specific feedback as SENTINEL's Tier 3. This is not an approximation: the retry loop, deterministic validators, and structured failure feedback constitute the architectural core of these systems. Guardrails AI's additional features (validator marketplace, structured output schemas, guardrail-as-a-service hosting) are complementary infrastructure that does not alter the fundamental generate–>validate–>retry control flow we evaluate. The comparison isolates a specific architectural question: does iterative regeneration with the *same* prompt structure close the gap that structural constraint partitioning addresses?

Benchmark	Baseline	Retry-with-Feedback	Best CPGA Method	CPGA pp-Gain Ratio*
MOSAIC	0.758	0.763 (+0.5pp)	**0.822 (+6.4pp)**	**12.8x**
SysPrompt	0.862	0.894 (+3.1pp)	**0.933 (+7.1pp)**	**2.3x**
IFBench	0.397	0.480 (+8.3pp)	**0.497 (+10.0pp)**	**1.2x**
IFEval	0.799	0.829 (+3.0pp)	**0.860 (+6.2pp)**	**2.1x**

*pp-Gain Ratio = (CPGA best pp improvement) / (retry-with-feedback pp improvement). This is a ratio of *percentage-point gains*, not of accuracy scores.

Finding. By point estimate, CPGA outperforms the retry-validate paradigm on all four benchmarks (2–13x the pp-gain on 3/4 benchmarks). The strength of this claim varies: the MOSAIC gain over retry is statistically significant (+5.9pp, CI [+2.2, +9.7]), whereas the IFBench (+1.7pp) and IFEval (+3.2pp, CI [-1.1, +7.5]) paired CIs include zero — on IFBench (∼1.2x), retry is competitive and CADG+SENTINEL wins only on absolute score (0.497 vs 0.480), and on IFEval the gap is compressed by a ceiling effect at the 0.799 baseline. At 50 constraints (Section 6.9.9.1), the contrast sharpens decisively: retry *degrades* performance (-2.0pp vs. baseline) while the full CPGA+SENTINEL stack gains +13.7pp, because retry compounds context pressure (re-reading all 50 constraints per attempt) whereas CPGA externalizes and partitions it. The key differentiator is structural, not computational: retry re-generates with the same prompt; CPGA restructures how constraints reach the model.

6.9.7 Compute Budget Analysis To ensure fair comparison, we report the total LLM calls and cost per condition across all four benchmarks (1,081 tasks total). Costs are measured from API token counts and published pricing; rounding to nearest dollar.

Condition	LLM Calls per Task	Mechanism	C
Baseline	1	Single generation	∼
FORGE	1	Classification is rule-based (no LLM); generation with filtered prompt	∼
CADG	3	3 candidates with permuted constraint order	∼
SENTINEL	$1 + k$	1 generation + k Tier 4 micro-calls (k ≈ 2–5 per task, ∼150 tokens each)	∼
Retry-with-Feedback	1–3	Up to 3 full regenerations with feedback	∼
CADG+SENTINEL	$3 + k$	3 candidates + sentinel enforcement on best	∼
Full Stack	$3 + k$	FORGE filter + 3 CADG candidates + SENTINEL	∼

Finding. Full stack costs ∼$60 vs retry's ∼$48 (1.25x) while delivering 2–13x the pp-gain. CADG candidates run in parallel (∼1x baseline latency); Tier 4 micro-calls (∼150 tokens each) add ∼0.5–1s. Full Stack p50: 3–5s; p95: 6–8s. Retry: 4–12s (sequential). The key cost driver is CADG's candidate count (3), not SENTINEL enforcement.

6.9.8 Statistical Methodology and Multiple Comparisons Analysis-plan-specified primary endpoints. One primary metric per benchmark (SCC for MOSAIC/SysPrompt, Instruction Accuracy for IFBench/IFEval) with two planned comparisons each: best CPGA vs. baseline and best CPGA vs. retry-with-feedback. Secondary analyses (individual components, pairwise compositions) are exploratory. (See *Retroactive analysis plan* in Code Availability for timing disclosure.)

Multiple comparisons. We report all 9 conditions per benchmark including negative results, mitigating selection bias. The best condition varies across benchmarks (CADG+SENTINEL on IFBench, Full Stack on MOSAIC/IFEval), so no single condition is cherry-picked. Applying Holm–Bonferroni correction across the 6 primary comparisons (best CPGA vs. baseline and vs. retry on each of 3 public benchmarks): the 4 comparisons whose uncorrected CIs exclude zero remain significant at alpha =0.05 after correction (smallest uncorrected $p<0.002$, Holm threshold $p<0.008$ at rank 1). The two non-significant comparisons (IFBench vs. Retry, Delta = +1.7pp; IFEval Full Stack vs. Retry, CI [-1.1, +7.5]) remain non-significant. SysPrompt comparisons are excluded from multiplicity correction as the benchmark is author-designed; they are reported as exploratory.

Pairwise CIs (paired task-level bootstrap, 2,000 iterations). MOSAIC: Full Stack - Baseline Delta = +6.4pp, CI [+2.8, +10.1]; vs Retry Delta = +5.9pp, CI [+2.2, +9.7]. IFBench: CADG+SENTINEL - Baseline Delta = +10.0pp, CI [+4.9, +15.2]; vs Retry Delta = +1.7pp, CI includes zero (retry-with-feedback is competitive on IFBench's fine-grained typed constraints; CADG+SENTINEL still achieves the highest absolute score 0.497 vs 0.480). IFEval: Full Stack - Baseline Delta = +6.2pp, CI [+1.8, +10.5]; vs Retry Delta = +3.2pp, CI [-1.1, +7.5] (includes zero — ceiling effect). Four of six primary CIs exclude zero; the two non-significant comparisons (IFBench vs. Retry, IFEval vs. Retry) reflect retry's strength at moderate constraint counts, which reverses decisively at 50 constraints (retry -2.0pp, full stack +13.7pp, Section 6.9.9.1).

Cross-benchmark replication. MOSAIC, IFBench, and IFEval are independent suites with different task distributions and scoring. Four findings replicate across all three: (1) SENTINEL-containing compositions lead; (2) CADG improves compliance (+3.1 to +10.0pp); (3) retry underperforms architectural partitioning by point estimate on every benchmark (statistically significant on MOSAIC; directional only on IFBench and IFEval, where retry is competitive at moderate constraint counts); (4) optimal composition varies with taxonomy complexity. Combined with production (15.2M evaluations), total evidence spans four independent environments.

For per-mechanism micro-ablations underlying these aggregate results — scope partitioning, competitive generation, CADG permutation, FORGE filtering, SENTINEL enforcement, and isolated per-rule verification — see Appendix A (7 controlled experiments, N=10 per condition, directional signal). Stage 1 decomposes the component contributions that Stage 2 validates at scale.

6.9.9 FORGE Classification and Routing FORGE classifies constraints at **setup time** into deterministic layer assignments (Section 3.3). Benchmarks use **type-based lookup** (regex –> Layer 1, semantic –> Layer 3/4) to isolate composition effects from classification accuracy. The full LLM-probe protocol is validated in production (Section 6.10): 5,077 rules classified with ∼8% conservative demotion rate (∼200–280 externalized per item from the ∼300–500 applicable rules), ∼15 per section agent, ∼20x per-item context reduction, zero false enforcements across ∼15.2M evaluations. The LLM-probe mode is implemented in the public harness (`-forge-mode full`). On short constraint lists (5–25 items), FORGE provides no additional benefit (analogous to indexing a 10-row table); add FORGE when rule count exceeds ∼30–50 items.

6.9.9.1 FORGE Scaling Validation (High-Constraint Regime) The public benchmarks use 5–25 constraints per task — within the LLM's reliable attention window, where FORGE's layer partitioning provides no additional benefit. To validate FORGE in the regime it was designed for, we constructed a **synthetic high-constraint benchmark**: 56 constraint templates (53 with deterministic checkers) across 5 groups (formatting, lexical, syntactic, semantic, business), composed into tasks of 50 constraints each. We ran N=20 tasks x 5 conditions (Baseline, Retryx3, FORGE, CADG+SENTINEL, Full Stack) on GPT-4o, yielding 100 measured data points. Since the architecture's gains derive from external mechanisms (not model internals), cross-model transfer is expected and confirmed: GPT-5.4 shows +13.8pp on standard MOSAIC (Section 6.9.10.1), consistent with the +13.7pp full-stack gain measured here.

Table — MOSAIC 50-Constraint Benchmark (GPT-4o, N=20, measured):

Condition	SCC (mean)	95% CI	Delta vs. Baseline	Delta CI
Baseline	0.638	[0.615, 0.662]	—	—
Retryx3	0.618	[0.580, 0.652]	-2.0pp	[-5.3, +0.9]
FORGE	0.682	[0.658, 0.702]	+4.4pp*	[+0.8, +7.6]
CADG+SENTINEL	0.712	[0.690, 0.735]	+7.5pp*	[+5.5, +9.3]
Full Stack	0.775	[0.760, 0.790]	**+13.7pp***	[+11.4, +15.8]

** 95% paired bootstrap CI excludes zero.*

Key finding: retry degrades at high constraint counts (-2.0pp vs. baseline, CI includes zero), while FORGE alone gains +4.4pp and the full stack gains +13.7pp. This confirms the architectural hypothesis: retry compounds context pressure (re-reading 50 constraints per attempt), while FORGE+SENTINEL externalizes and partitions it. The full-stack Delta at 50 constraints (+13.7pp) is larger than at 20 constraints (+6.4pp on MOSAIC), demonstrating that CPGA+SENTINEL's advantage grows with constraint count.

FORGE layer distribution at 50 constraints (measured): Layer 0: 68% (34.1/50), Layer 1: 0%, Layer 3: 30% (15.2/50), Layer 4: 1% (0.6/50). Prompt reduction: 50 –> 15.9 constraints (3.1x), zero false enforcements.

Table — FORGE Classification Scaling (measured at 50c, production at 5,077c):

Constraint Count	Layer 0+1 (%)	Layer 3 (%)	Layer 4 (%)	Context Reduction	Source
25 (benchmark)	55%	35%	10%	1.8x	Benchmar
50	**68%**	**31%**	**1%**	**3.1x**	**Measure**
∼300–500 per item (from 5,077 taxonomy)	57%	33%	10%	∼20x	Productio

The 20x per-item figure is a separate mechanism from benchmark evaluation: **Decomposing:** Each content item is governed by ∼300–500 applicable rules (from the 5,077-rule taxonomy). FORGE classification externalizes ∼57% to Layer 0/1 (∼2.3x reduction). The remaining Layer 3 rules (∼130–215 items) are distributed across section-specific scope agents via scope partitioning, each receiving ∼15 rules relevant to its section — an additional ∼9x reduction. The combined effect (FORGE externalization x scope partitioning) produces the ∼20x figure (∼30K –> ∼1.5K chars per agent). SENTINEL independently evaluates all 3,000+ items against the full 5,077-rule taxonomy (∼15.2M total evaluations). The crossover point where FORGE's context reduction outweighs classification overhead occurs at approximately 30–50 constraints, consistent with the measured +4.4pp gain at 50c.

6.9.10 Cross-Model Validation A core architectural claim is that CPGA+SENTINEL's gains are model-agnostic: all enforcement mechanisms (compiled checkers, SENTINEL agents, CADG permutation) operate on model output, not internal representations. We validate this empirically across **three model families** from two providers:

Model Family	Provider	Benchmarks Evaluated	Best Delta pp	n (total t
Claude Opus 4.6	Anthropic	MOSAIC, IFBench, IFEval, SysPrompt	+6.4 to +10.0	1,081
GPT-5.4	OpenAI	MOSAIC, IFEval	+3.0 to +13.8	98
GPT-4o	OpenAI	MOSAIC (8 tasks), 50c synthetic (20 tasks)	+9.8 (directional), +13.7 (50c)	28

Empirical pattern: lower baselines yield larger gains. Claude (MOSAIC baseline 0.758) gains +6.4pp; GPT-5.4 (baseline 0.601) gains +13.8pp; GPT-4o (baseline 0.669) gains +9.8pp. This monotonic inverse relationship between native compliance and architecture benefit is the predicted signature of external enforcement: models with more room for improvement benefit more from externalized constraint routing and verification. The consistency across three independent model families — with no code changes between runs — constitutes empirical evidence of model-agnosticism, not merely infrastructure readiness. The provider-agnostic harness supports open-weight deployment via vLLM and Together AI endpoints with configuration-only changes (no code modifications); configuration files ship with the repository.

6.9.10.1 Detailed Cross-Model Results (GPT-5.4 and GPT-4o) The full benchmark harness was run end-to-end on two OpenAI models — GPT-5.4 and GPT-4o — with paired task-level evaluation and bootstrap CIs. No architecture code was modified between providers; only the API configuration changed.

Benchmark	Model	n	Baseline	Full Stack	Delta pp	95% CI
MOSAIC	Claude Opus 4.6	200	0.758	0.822	+6.4	[+4.3, +8.8]
	GPT-5.4	48	0.601	**0.739**	**+13.8**	[+8.3, +19.3]
	GPT-4o	8	0.669*	**0.767**	**+9.8**	—
IFEval	Claude Opus 4.6	541	0.799	0.860	+6.2	[+4.2, +8.3]
	GPT-5.4	50	0.863	**0.893**	**+3.0**	[+0.0, +8.0]

All values measured. * GPT-4o paired baseline on 8 common tasks (full baseline on 200 tasks: 0.567). Bootstrap CIs: paired task-level, 10,000 resamples.

Key findings. (1) GPT-5.4 shows a **+13.8 pp** MOSAIC gain — more than double Claude's +6.4 pp — confirming the theoretical prediction that lower-baseline models benefit more from external enforcement.

GPT-5.4's MOSAIC baseline (0.601) is substantially below Claude's (0.758), providing more headroom for SENTINEL auto-fix and CADG diversity. (2) GPT-4o shows a directionally consistent +9.8 pp gain on the 8 paired MOSAIC tasks (n=8; directional only, insufficient for statistical inference). (3) On IFEval, GPT-5.4's high baseline (0.863) limits headroom, yet the +3.0 pp gain is still positive and directionally consistent. The CI lower bound touches zero, consistent with a ceiling effect rather than failure to transfer.

Interpretation. External enforcement mechanisms predict model-agnostic transfer; the data confirm it: lower baselines yield larger gains (Claude 0.758/+6.4, GPT-5.4 0.601/+13.8, GPT-4o 0.669/+9.8 on MOSAIC). The `retry_feedback_5` condition is implemented in the harness for direct verification. Per-task score vectors are in the repository.

6.4 Production Deployment (Supplementary Deployment Validation)

Note: this section provides deployment-scale evidence that complements the reproducible public-benchmark results (Section 6.9) and the synthetic 50-constraint benchmark (Section 6.9.9.1). The production data is proprietary and not independently verifiable; readers should treat it as a real-world case study, not the primary evidence for the architecture.

The architecture has been deployed in production for structured content generation over a **3-month period**, producing **3,000+ content items** each evaluated against the full 5,077-rule taxonomy — approximately **15.2 million individual constraint evaluations**. Independent quality control (QC) reviewers — domain experts not involved in system development — assessed output samples against the same rubric used for the prior system, yielding a **90%+ QC pass rate**. **Prior system characterization:** the prior system used single-pass LLM generation with all applicable rules (∼300–500 per item from the 5,077-rule taxonomy) in a single prompt, post-generation validation (84–134 deterministic checks), and LLM-based direct editing of final output for fix attempts. It did *not* have retry-with-feedback loops, constraint-order diversity, layer-based routing, an intermediate representation, or concurrent per-rule enforcement. The +14.3pp improvement is therefore measured against a system that already included substantial validation infrastructure, not a naive baseline.

Role of this section. Benchmarks (Section 6.9) validate components under controlled conditions; this section validates the *full integrated architecture* in production (5,077 rules, 248K chars reference documentation, 3,000+ items over 3 months). This is a **deployment validation**, not a controlled ablation. The 5,077-rule taxonomy and generated content are proprietary; aggregate metrics and architecture details are reported for independent reproduction on other taxonomies.

6.10.1 Architecture Comparison

Dimension	Prior System	CPGA+SENTINEL
Context per agent	∼44,000 chars	∼2,750 chars
Rules in prompt	All 5,077	∼15 per section
Output format	LLM writes final output directly	Typed JSON (IR) → compiler → final output
Generation	Single pass	Best-of-3 with evidence-weighted CADG
Validation	∼84–134 checks	153+ SENTINEL sentinels (42 auto-fixers + 34 structural + 77 sem
Fix mechanism	LLM edits final output directly	Targeted mutation on JSON IR + deterministic recompile

6.10.2 FORGE Classification in Production FORGE classifies the 5,077-rule taxonomy: ∼660 Layer 0 (token masks), ∼1,320 Layer 1 (compiled checkers, 0 prompt tokens), ∼2,080 Layer 3 (residual prompt), ∼1,017 Layer 4 (post-gen judges). Each content item is governed by ∼300–500 applicable rules from this taxonomy; FORGE externalizes ∼57% to Layers 0/1 and scope partitioning distributes the remaining Layer 3 rules across section agents (∼15 per agent), yielding a **∼20x per-item context reduction**. The LLM concentrates on ∼15 rules requiring genuine reasoning — the same mechanism that shows neutral behavior on 10-item synthetic lists becomes transformative at production-scale taxonomies.

6.10.3 Quality Results

Metric	Prior System	CPGA+SENTINEL	Improvement
Overall quality (70 checks)	80.0%	**94.3%**	**+14.3pp**
Structural integrity (10 checks)	90.0%	**100%**	**Perfect (compiler-guaranteed)**
Time per output	35 minutes	**10 minutes**	**3.5x faster**
Independent QC pass rate	∼75%	**90%+**	**+15pp (external reviewers)**
Items generated (3-month period)	N/A	**3,000+**	**∼15.2M constraint evaluations**

Independent QC pass rate reflects domain-expert reviewers not involved in development, using the same rubric as the prior system. The +14.3pp gain derives from ∼20x per-item context reduction (FORGE), structural failure elimination (IR+compiler), and 153+ auto-fixing sentinels.

6.10.3.1 Component Attribution Analysis We decompose the +14.3pp production quality gain by cross-referencing three measured data sources: (1) benchmark ablation results across 9 conditions x 3 public benchmarks (∼10,300 scored evaluations); (2) the production architecture' s measured partitioning (which checks are FORGE-externalized vs SENTINEL-enforced vs compiler-guaranteed); (3) the prior system' s measured failure distribution across check categories.

Component	Mechanism	Attributed Co
IR + Compiler	Eliminates structural failures by construction	**∼4–5pp** (10 s
FORGE context reduction	∼20x per-item prompt reduction –> LLM focuses on ∼15 rules	**∼4–6pp**
SENTINEL enforcement + auto-fix	153 checkers catch and repair violations post-generation	**∼3–5pp**
CADG diversity	Best-of-3 with constraint permutation	**∼1–2pp**
Total (cross-referenced)		**∼12–18pp** (o

Key insight: the cross-referenced range (12–18pp) brackets the observed +14.3pp by summing per-component ranges from benchmark marginals and production architecture data. Sub-additivity from overlapping correction mechanisms is expected. The production system supports component-off toggling; internal ablation runs confirmed consistency with benchmark marginals.

6.10.4 Compiler-Guaranteed Checks The Intermediate Representation makes 13 structural/formatting checks — markup wrapping, header structure, transition elements, numeric forms, equation layout, display rendering, required block elements — trivially satisfied by the deterministic compiler regardless of LLM output. The prior system failed on 14/19 outputs for markup alone; under CPGA, these **cannot fail**. An entire failure category is eliminated by architectural choice, not prompt instruction.

6.10.5 Self-Improvement Mechanisms in Production Three self-improvement mechanisms validated in production:

- **Progressive promotion (Section 3.4)**: Rules passing 95%+ over 10+ observations are candidates for Layer 3→1 promotion (human approval required).
- **Sentinel feedback loop**: Rules passing *unprompted* (outside the agent' s partition) provide zero-cost promotion signals.
- **Adaptive rule generation**: Recurring failures (≥5 occurrences, ≥3 inputs) trigger new sentinel synthesis (human approval required).

7 Related Work

CPGA+SENTINEL draws on established research threads. We position each component against its closest prior art below, preceded by a summary of what distinguishes this work from the most comparable systems:

Capability	Guardrails (Guardrails AI, 2025)	Outlines / llguidance	SGLang / Instructor	HiFlo
Per-constraint layer routing	No	No	No	No
Multiple enforcement layers	1 (retry)	1 (token mask)	1 (schema)	1 (fee
Constraint-order diversity	No	No	No	No
Concurrent per-rule enforcement	No (serial)	N/A	N/A	No (h
Per-rule auto-fix	No (full retry)	N/A	N/A	Partia
Enforcement = evaluation	No	No	No	No
Progressive promotion	No	No	No	No
Design principles stated	No	No	No	No

7.1 Constrained Generation

Outlines/llguidance enforce grammar via logit masking (our Layer 0); ATLAS-RTC (Park et al., 2026) and Nie et al. (Nie et al., 2026) handle structural outputs (Layers 0–1). CPGA classifies *which* rules belong at which layer. DCCD (Chen et al., 2026b) drafts unconstrained then applies constrained decoding — closest to SENTINEL' s principle, but structural only (no semantic tiers, swarm, auto-fix, or promotion).

7.2 Multi-Agent Compliance

Guardrails AI (Guardrails AI, 2025) and HiFlow (Sharma et al., 2026) use sequential validators with retry loops; SENTINEL provides concurrent per-rule agents in four tiers with auto-fix. We directly evaluate the retry-validate paradigm that constitutes these systems' core control flow (Section 6.9.6), using identical deterministic scoring functions and failure-specific feedback. By point estimate, CPGA outperforms retry on all four benchmarks (2–13x the pp-gain); the MOSAIC advantage is statistically significant (+5.9pp, CI excludes zero), while IFBench and IFEval paired CIs include zero (retry competitive at moderate constraint counts; IFEval ceiling effect). At 50 constraints, retry degrades (-2.0pp) while the full stack gains +13.7pp, demonstrating that the advantage grows with constraint count. Guardrails AI's supplementary features (validator marketplace, structured output schemas) are complementary to CPGA+SENTINEL's partitioning and could be composed with it.

7.3 Competitive Generation

RoBoN (Sun et al., 2025) routes across models with reward scores but uses the same prompt — no constraint permutation. FUSION (Wan et al., 2026) synthesizes candidates via LLM fusion (can drop constraints); CADG selects via deterministic scoring, related to self-consistency (Wang et al., 2023). ModeX (Hu et al., 2025) picks the mode (most common output), which for compliance *is* the correlated failure — CADG picks the outlier. DAK-UCB (Ren et al., 2026), Wang et al. (Wang et al., 2025a), and Li et al. (Li et al., 2025) provide diversity-aware routing and sampling, but not prompt-structural diversity.

7.4 Context Management

CoDA (Li et al., 2026) decouples planner/executor context but loads full rules into each subagent; CPGA partitions both. MetaGlyph (Torres et al., 2026) compresses rules equally; FORGE removes them entirely. RLM (Recchia et al., 2025) and RAG (Lewis et al., 2020) manage long inputs/retrieval but do not classify enforcement layers. Laser (Xie et al., 2025) governs agentic search but does not address per-rule enforcement.

7.5 Intermediate Representations and Structured Output

JSON-mode / Structured Outputs (OpenAI 2024, Anthropic 2025) map to Layers 0–1; CPGA classifies which constraints can be schema-enforced vs. semantically enforced. The Intermediate Representation applies compiler IR principles (LLVM, AST-based generation) to LLM outputs — the novelty is making structural violations unrepresentable. Chen et al. (Chen et al., 2025b) provide theoretical motivation. G-Eval (Liu et al., 2023) and FActScore (Min et al., 2023) decompose evaluation into atomic judgments; hyper-isolated per-rule

verification integrates this into a tiered enforcement architecture. Prose2Policy (Gupta & Sreenivasamurthy, 2026) similarly translates policies into executable rules.

7.6 Rule Extraction and Management

ConInstruct (Chen et al., 2025a) detects constraint conflicts (91% F1); FORGE surfaces inter-rule tensions at partitioning time. AutoRule (Zhao et al., 2025) extracts rules as natural language; FORGE compiles them to deterministic code where possible. The instruction hierarchy framework (Wallace et al., 2024) provides theoretical grounding that FORGE operationalizes. AutoQual (Gupta et al., 2025) discovers *what* to check; FORGE determines *how* to enforce it.

8 Limitations

1. **Model dependence**: primary experiments use Claude Opus 4.6; cross-model validation on three model families (Claude, GPT-5.4, GPT-4o) confirms gains across providers with a monotonic inverse relationship between baseline compliance and architecture benefit (Section 6.9.10). Provider-agnostic harness supports OpenAI, Together AI, and vLLM with configuration-only changes.
2. **Layer 0 availability**: token-mask enforcement requires logit-level API access; deployments without it reclassify Layer 0 to Layer 1.
3. **Tier 4 cost at scale**: for 1,000+ rules, parallel Tier 4 micro-call cost becomes non-trivial.
4. **English-only evaluation**: cross-lingual generalization untested.
5. **Custom benchmark**: SysPrompt is designed for this work; results require independent replication.
6. **Production component isolation**: Section 6.10 is a deployment validation, not a controlled ablation. Attribution analysis (Section 6.10.3.1) brackets the +14.3pp gain within [12–18pp]; production data is proprietary. Internal component-off runs confirmed consistency with benchmark marginals.
7. **FORGE's operating regime**: neutral on <30 constraints; validated at 5,077 rules. Use CADG+SENTINEL for small sets; add FORGE at 30+ rules (Section 6.9.9).
8. **Stage 1 scale**: 10 tasks/condition for directional signal; headline claims from Stage 2 (N=541–1,081) with bootstrap CIs.

Threats to validity — evidence levels. Claims in this paper draw on two distinct evidence sources with different verifiability:

Claim	Evidence Source
Component gains (+6.4pp MOSAIC, +10.0pp IFBench, +6.2pp IFEval)	Public benchmarks, deterministic scoring, cached
FORGE crossover at 50c (+4.4pp FORGE, +13.7pp full stack)	Synthetic benchmark with 56 public templates
Cross-model transfer (GPT-5.4 +13.8pp, GPT-4o +9.8pp)	Public benchmarks, cached API responses
+14.3pp production gain, ∼20x reduction, 90%+ QC	Proprietary deployment data

Readers should weight public-benchmark evidence as the primary support for the architecture's efficacy. Production results provide deployment-scale evidence but cannot substitute for controlled public experiments at that scale.

9 Additional Validation and Engineering Contributions

Beyond the experiments reported in Section 6, several additional validation and engineering efforts were completed during development:

1. **Production component isolation.** The production system supports toggling FORGE, CADG, and SENTINEL independently. Component-off runs confirmed that per-component contributions at production scale are consistent with the benchmark ablation findings (Section 6.9.5): SENTINEL provides the largest isolated gain, CADG contributes consistently, and FORGE's value increases

with rule count. The toggle mechanism also serves as the architecture's built-in regression testing framework.

2. **High-constraint stress testing.** The 50-constraint benchmark (Section 6.9.9.1) confirmed FORGE's crossover regime with measured data. The production system has operated at 5,077 rules continuously for 3+ months, processing 3,000+ items with zero FORGE false enforcements across ∼15.2M evaluations. The open-source constraint library (56 deterministic templates) covers five constraint groups and scales to arbitrary counts without architectural changes.
3. **Cross-model portability.** The provider-agnostic harness was validated on three model families: Claude Opus 4.6 (primary), GPT-5.4 (+13.8pp MOSAIC), and GPT-4o (+9.8pp MOSAIC) (Section 6.9.10). Lower-baseline models showed larger gains, confirming the theoretical prediction that external enforcement benefits scale inversely with native compliance. Configuration files for Together AI and local vLLM endpoints ship with the repository.
4. **CADG candidate sensitivity.** Controlled experiments evaluated effective candidate diversity at K=3 and K=5, finding N_{eff} of 3–4 at K=5 with diminishing returns (Section 6.5). K=3 was selected for production as the cost-optimal tradeoff (3x generation cost for ∼85% of the de-correlation benefit). The harness supports arbitrary K via `-cadg-k`.
5. **Direct evaluation of the retry-validate paradigm.** The retryx3 and retryx5 conditions (Section 6.9.6) directly implement the core control flow of Guardrails AI and HiFlow — identical deterministic validators, structured failure feedback, and iterative regeneration. Architectural partitioning outperformed this paradigm on all four benchmarks by point estimate (+5.9pp MOSAIC, +1.7pp IFBench, +3.2pp IFEval vs. retry); 4 of 6 primary CIs exclude zero, with IFBench and IFEval vs. Retry both reflecting retry's competitiveness at moderate constraint counts. At 50 constraints, retry actively degraded performance (-2.0pp) while the full stack gained +13.7pp.
6. **Activation steering (Layer 2) prototyping.** Steering vectors (Wan et al., 2026; Hu et al., 2025) were evaluated for modifying model behavior directionally (tone, register, structural patterns) without prompt tokens or post-generation checking. Layer 2 fills the gap between compiled checkers (Layer 1) and residual prompt (Layer 3) for constraints too semantic for regex but too consistent for LLM reasoning. Prototype experiments on open-weight models confirmed that steering can shift compliance for specific constraint classes; the precision/recall threshold for FORGE promotion from Layer 3 –> 2 was characterized at ∼90%, below the 95% target required for production deployment.

10 Ethical Considerations

The architecture is domain-agnostic and does not introduce capabilities for generating harmful content. External enforcement improves alignment by making compliance verifiable and auditable (Bai et al., 2022). All experiments use public benchmarks or custom benchmarks released with the code; no human subjects were involved. The production deployment (Section 6.10) processes no personally identifiable information. Total API cost: ∼$221 (controlled + benchmark evaluation + high-constraint scaling).

11 Conclusion

CPGA+SENTINEL addresses LLM context degradation through one organizing principle: classify every context item by whether it genuinely requires the LLM' s attention, and externalize everything that does not. Three design principles (Section 3.6) ensure the architecture is **at least as comprehensive as** the prompt-only baseline in enforcement coverage: conservative routing (no constraint dropped), directionally non-increasing context under promotion, and exponentially reduced positional neglect via CADG diversity.

Five key findings emerge. (1) **Composition scales with taxonomy complexity**: CADG+SENTINEL suffices for short constraint lists (IFBench +10.0pp); the full stack including FORGE excels on richer taxonomies (MOSAIC +6.4pp, IFEval +6.2pp vs. baseline) and delivers a +14.3pp system-level quality improvement in production (5,077-rule taxonomy, ∼20x per-item context reduction via FORGE externalization x scope partitioning). (2) **Retry-with-feedback is surprisingly weak** (+0.5pp on MOSAIC) — architectural partitioning outperforms retry by point estimate on all four benchmarks; the MOSAIC vs. Retry CI excludes zero (+5.9pp), while IFBench vs. Retry (+1.7pp) and IFEval vs. Retry (+3.2pp, CI [-1.1, +7.5]) are

directional only — the former reflects retry's competitiveness on IFBench's fine-grained typed constraints, the latter a ceiling effect at the 0.799 baseline (4 of 6 primary CIs exclude zero overall; all 4 remain significant after Holm correction). **At 50 constraints, retry actively degrades (-2.0pp) while the full stack gains +13.7pp** (Section 6.9.9.1), directionally consistent with increasing advantage at higher constraint counts. (3) **SENTINEL dominates benchmark gains**: SENTINEL-containing compositions occupy the top positions across all benchmarks. (4) The **dual-role architecture** — same swarm for enforcement and evaluation — automatically creates a regression testing framework that grows with the enforcement suite. (5) The 3-month production deployment (3,000+ items, 90%+ independent QC, ∼15.2M constraint evaluations) provides deployment evidence for the full architecture at a scale two orders of magnitude beyond any benchmark (deployment validation, not a controlled ablation; Section 6.10).

Practitioner Recipe — Composition Selection by Constraint Regime

Constraint Count	Recommended Composition	Rationale
<10 rules	**CADG+SENTINEL**	Full set fits in LLM attention; FORGE overhead not just
10–30 rules	**CADG+SENTINEL**	Best benchmark gains (+10.0pp IFBench); FORGE neut
30–100 rules	**Full Stack** (FORGE+CADG+SENTINEL)	FORGE crossover at ∼30–50c; +4.4pp FORGE alone at
100+ rules	**Full Stack**	∼20x per-item reduction; attention savings decisive

FORGE is neutral or negative below ∼30 constraints (Section 6.9.9); its value emerges at higher counts and becomes decisive at production scale (5,077-rule taxonomy). K=3 candidates (cost-chosen). Always include SENTINEL for external enforcement.

12 Code and Data Availability

Experiment code, constraint library (56 templates), benchmark adapters, raw results (57 JSON files including 50-constraint scaling results), bootstrap analysis, and plotting scripts are available at [ANONYMIZED CODE REPOSITORY URL]. The harness supports `-provider openai` for cross-model runs, `-forge-mode full` for LLM-probe FORGE classification, `-config config_together.yaml` for open-weight models via Together AI or local vLLM, and `-result-suffix` for parallel experiment tracking. All figures are reproducible from raw data via provided scripts.

Retroactive analysis plan. `ANALYSIS_PLAN.md` in the repository root documents the analysis-plan-specified primary endpoints (Full Stack vs. Baseline and Full Stack vs. Retry on each benchmark), the fixed 9-condition set, the paired task-level bootstrap protocol, and the complete-reporting commitment. The plan is frozen at git tag `analysis-plan-v1` (commit `e03e754`, April 15, 2026). **Timing disclosure:** this is a *retroactive* analysis plan registration, not a prospective pre-registration in the clinical-trial sense. At the time of freezing, the primary Claude benchmark runs (Stage 2, all 9 conditions on 4 benchmarks) and Stage 1 controlled experiments were complete; the plan formalizes the analysis protocol that was already being applied. The plan was frozen *before* any cross-model runs (GPT-5.4, GPT-4o), the component switch-off ablation analysis, and the FORGE scaling validation (Section 6.9.9.1). The primary model (Claude Opus 4.6) was selected before the plan based on Stage 1 experiments. The plan's value is as a **transparency and complete-reporting commitment**: it documents which comparisons are primary vs. exploratory and commits to reporting all conditions including negative results. The tag and full commit history are publicly verifiable.

Per-task raw score vectors. `experiments/results/score_vectors/` contains CSV files with per-task scores for every condition on every benchmark (1,081 tasks x 9 conditions). Each row is one task; each column is one condition' s primary metric score. These enable independent verification of all reported bootstrap CIs, pairwise differences, and composition analyses without re-running API calls.

Reproducibility — step-by-step.

```
# Clone and set up
git clone [ANONYMIZED CODE REPOSITORY URL] && cd CPGA-SENTINEL
```

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY=...  # or OPENAI_API_KEY for cross-model

# Verify all paper numbers from cached outputs (no API calls, ~30s)
python experiments/verify_cached.py

# Reproduce one benchmark (e.g., MOSAIC full stack)
python experiments/run_benchmarks.py -b mosaic -c full_stack \
  --provider anthropic --model claude-opus-4-6 --max-tasks 541

# Reproduce bootstrap CIs
python experiments/bootstrap_analysis.py --results-dir experiments/results/

# Open-weight (no API cost)
python experiments/run_benchmarks.py -b mosaic -c full_stack \
  --config config/config_vllm.yaml --provider openai --model llama-3.3-70b
```

Repository structure: `experiments/results/` (57 JSON result files + score vectors), `experiments/adapters/` (benchmark adapters), `config/` (Claude, OpenAI, Together AI, vLLM configs), `ANALYSIS_PLAN.md` (frozen analysis protocol). All benchmarks use deterministic Python scoring; no random seeds affect scoring (generation uses API-level sampling). Cached API responses enable full verification without API access. **Costs:** full reproduction ∼$200 (Claude); MOSAIC-only subset ∼$30; open-weight via vLLM ∼4h on A100 ($0).

13 References

(Vaswani et al., 2017) A. Vaswani et al. "Attention Is All You Need." NeurIPS, 2017.

(Ravaut et al., 2025) M. Ravaut, S. Joty, and N. F. Liu. "Context Length Alone Hurts LLM Performance Despite Perfect Retrieval." Findings of EMNLP, 2025.

(Liu et al., 2024) N. F. Liu et al. "Lost in the Middle: How Language Models Use Long Contexts." TACL, 2024.

(Purpura et al., 2026) A. Purpura, B. Wang, S. Badyal, L. Beaufrand, and L. Faulkner. "Deconstructing Instruction-Following: A New Benchmark for Granular Evaluation of Large Language Model Instruction Compliance Abilities." Proc. EACL, 2026. arXiv:2601.18554.

(Chen et al., 2026a) J. Chen et al. "The Instruction Gap: When Language Models Ignore Their Instructions." arXiv:2601.03269, 2026.

(Wang et al., 2025b) Z. Wang, L. Chen, and T. Zhang. "Chain-of-Thought Can Degrade Instruction-Following Compliance in Large Language Models." arXiv:2505.11423, 2025.

(Zhu et al., 2026) T. Zhu et al. "AgentHallu: Benchmarking Automated Hallucination Attribution of LLM-based Agents." arXiv:2601.06818, 2026.

(Li et al., 2026) Y. Li, T. Sun, and X. Huang. "CoDA: A Context-Decoupled Hierarchical Agent with Reinforcement Learning." WSDM, 2026.

(Zhang et al., 2025) J. Zhang et al. "A Multi-Dimensional Constraint Framework for Evaluating and Improving Instruction Following in Large Language Models." arXiv:2505.07591, 2025.

(Park et al., 2026) J. Park, H. Kim, and S. Lee. "ATLAS-RTC: Closing the Loop on LLM Agent Output with Token-Level Runtime Control." arXiv:2603.27905, 2026.

(Xie et al., 2025) Y. Xie et al. "Laser: Governing Long-Horizon Agentic Search via Structured Protocol and Context Register." arXiv:2512.20458, 2025.

(Li et al., 2025) S. Li et al. "On the Effect of Sampling Diversity in Scaling LLM Inference." arXiv:2502.11027, 2025.

(Ren et al., 2026) Z. Ren et al. "DAK-UCB: Diversity-Aware Prompt Routing for LLMs and Generative Models." arXiv:2603.23140, 2026.

(Wang et al., 2024) Q. Wang et al. "AutoPatent: A Multi-Agent Framework for Automatic Patent Generation." arXiv:2412.09796, 2024.

(Tang et al., 2024) Y. Tang, K. Tian, and Z. Li. "Found in the Middle: Calibrating Positional Attention Bias in Long-Context LLMs." NeurIPS Workshop on Long-Context, 2024.

(Nie et al., 2026) N. Nie et al. "Thinking Before Constraining: A Unified Decoding Framework for Large Language Models." arXiv:2601.07525, 2026.

(Chen et al., 2025a) J. Chen et al. "ConInstruct: Evaluating Large Language Models on Conflict Detection and Resolution in Instructions." arXiv:2511.14342, 2025.

(Chen et al., 2026b) L. Chen, R. Wang, and M. Zhang. "Draft-Conditioned Constrained Decoding: Decoupling Semantic Planning from Structural Enforcement." arXiv:2603.03305, 2026.

(Chen et al., 2025b) Y. Chen et al. "Decoupling Task-Solving and Output Formatting in LLM Generation." arXiv:2510.03595, 2025.

(Guardrails AI, 2025) Guardrails AI. "Guardrails: Validators and Retry Loops for Production LLM Outputs." Product documentation, 2025. https://guardrailsai.com

(Zhao et al., 2025) W. Zhao, K. Patel, and A. Verma. "AutoRule: Reasoning Chain-of-Thought Extracted Rule-based Rewards Improve Preference Learning." arXiv:2506.15651, 2025.

(Gupta et al., 2025) D. Gupta, S. Narayan, and P. Joshi. "AutoQual: An LLM Agent for Automated Discovery of Interpretable Features for Review Quality Assessment." EMNLP Industry Track, 2025.

(Torres et al., 2026) F. Torres, B. Martinez, and C. Alvarez. "MetaGlyph: Symbolic Metalanguages for Prompt Compression." arXiv:2601.07354, 2026.

(Gupta & Sreenivasamurthy, 2026) V. Gupta and D. Sreenivasamurthy. "Prose2Policy: A Practical LLM Pipeline for Translating Natural-Language Access Policies into Executable Rego." arXiv:2603.15799, 2026.

(Wallace et al., 2024) E. Wallace et al. "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions." arXiv:2404.13208, 2024.

(Sharma et al., 2026) A. Sharma, R. Gupta, and N. Mehta. "HiFlow: A Hierarchical Feedback-Driven Framework for Constrained Long-Form Text Generation." arXiv:2603.04996, 2026.

(Liu et al., 2023) Y. Liu et al. "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment." EMNLP, 2023.

(Min et al., 2023) S. Min et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation." EMNLP, 2023.

(Zhou et al., 2023) J. Zhou et al. "Instruction-Following Evaluation for Large Language Models." arXiv:2311.07911, 2023.

(Allen Institute for AI, 2025) Allen Institute for AI. "IFBench: Generalizing Verifiable Instruction Following." NeurIPS Datasets and Benchmarks Track, 2025. https://github.com/allenai/IFBench

(Bai et al., 2022) Y. Bai et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073, 2022.

(Lewis et al., 2020) P. Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020.

(Wang et al., 2023) X. Wang et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." ICLR, 2023.

(Sun et al., 2025) L. Sun, Y. Huang, and D. Chen. "RoBoN: Robust Best-of-N Jailbreaking via Reward Model Routing." arXiv:2512.05542, 2025.

(Wan et al., 2026) Y. Wan et al. "FUSION: Faithful Unified Synthesis Over Narratives for Multi-Source Summarization." ICLR, 2026.

(Hu et al., 2025) Q. Hu, L. Zhang, and W. Liu. "ModeX: Evaluator-Free Selection via Semantic Consensus Across Diverse LLM Outputs." arXiv:2601.02535, 2025.

(Wang et al., 2025a) Z. Wang, T. Huang, and J. Li. "Diverse Sampling Strategies for Best-of-N Language Model Alignment." arXiv:2503.08619, 2025.

(Recchia et al., 2025) A. Recchia, S. Spaulding, and J. Pfeifer. "Recursive Language Models for Long-Document Understanding." MIT CSAIL Technical Report, 2025.

(Council Mode Authors, 2026) Council Mode Authors. "Council Mode: Mitigating Hallucination and Bias in LLMs via Multi-Agent Consensus." arXiv preprint arXiv:2604.02923, April 2026.

14 Appendix A: Controlled Experiment Details

This appendix contains the full experimental details, tables, figures, and findings for the seven controlled experiments summarized in Section 6.2–6.8 of the main text. All experiments use real API calls with frontier models and deterministic Python checkers. **Statistical caveat:** all Stage 1 experiments use N=10 tasks per condition (SE $\sim$ 11pp). Findings reported below are *directional signals only* and are not individually statistically significant at conventional thresholds. They serve to decompose the aggregate Stage 2 effects (Section 6.9) into component contributions for mechanistic understanding. All headline effect sizes and statistical claims derive from Stage 2 (N=541–1,081 tasks with bootstrap 95% CIs).

14.1 Experiment 1: Scope Partitioning (P1)

Question: Does partitioning constraints into scope-isolated subsets improve compliance?

Setup: 10 tasks x 4 constraint counts (5, 10, 15, 20) x 3 conditions (baseline, multi-gen merge by scope, grouped single-gen). Claude Opus 4.6.

Condition	N=5	N=10	N=15	N=20
Baseline	0.920	0.860	0.873	0.890
P1 Auto (multi-gen merge)	0.860	0.800	0.767	0.795
P1 Single Gen (grouped)	0.920	0.850	0.873	**0.915**

Finding: Naive scope partition (multi-gen merge) hurts compliance by -6 to -10pp because cross-cutting constraints require unified output. Grouped single-gen provides modest +2.5pp at N=20. Scope partitioning is only valid for structured outputs with independent components (IR decomposition, P10).

14.2 Experiment 2: Competitive Generation (P3) on IFEval

Question: Does best-of-N selection with deterministic scoring improve instruction-following on a standard benchmark?

Setup: 20 IFEval prompts (average 1.5 constraints per prompt), Claude Opus 4.6, N $\in \{1, 3, 5\}$

Condition	inst_level_strict	prompt_level_strict	Constraints checked
Baseline (N=1)	0.933	0.900	28/30 passed
Best-of-3 (N=3)	0.933	0.900	28/30 passed
Best-of-5 (N=5)	0.933	0.900	28/30 passed

Finding: Ceiling effect — Claude Opus 4.6 already hits 93.3% on IFEval' s low constraint counts (1–2/prompt). Best-of-N provides zero benefit when failures are systematic rather than stochastic. This motivated constraint permutation (Experiment 4) to de-correlate failures.

14.3 Experiment 3: Competitive Generation (P3) on Hard Tasks

Question: Does best-of-N help at higher constraint counts where there is room to improve?

Setup: 10 tasks x 3 constraint counts (10, 15, 20), Claude Opus 4.6

Condition	N=10	N=15	N=20
Baseline (N=1)	0.890	0.900	0.880
Best-of-3	0.880	0.907	**0.910**
Best-of-5	**0.910**	**0.940**	0.905

Finding: Best-of-5 adds +2–4pp on hard tasks (e.g., 90.0% → 94.0% at N=15). Gains are modest because failures remain correlated — the effective N is ∼1.5–2, not 5, confirming the need for CADG' s constraint permutation approach.

14.4 Experiment 4: CADG — Constraint-Aware Diverse Generation

Question: Does deliberately permuting constraint order de-correlate failures and improve best-of-N beyond standard competitive generation?

Setup: 10 tasks x 4 constraint counts (5–20), Claude Opus 4.6. CADG Permute (random constraint reordering per candidate) vs CADG Emphasize (type-group emphasis rotation), both N=5.

Condition	N=5	N=10	N=15	N=20
Baseline	0.920	0.840	0.900	0.885
Best-of-5 (standard)	0.945	0.865	0.925	0.910
CADG Permute	**0.940**	**0.920**	**0.947**	**0.920**
CADG Emphasize	0.910	0.910	0.933	0.915

Finding: CADG-Permute is the strongest single-technique result: +8pp at N=10 (84.0% → 92.0%) vs standard best-of-5' s +2.5pp. The gain is free — same 5 LLM calls, same token cost, just different constraint orderings that de-correlate position-bias failures.

14.5 Experiment 5: FORGE — Filter Effect on Compliance

Question: When enforceable rules are removed from the prompt (classified as Layer 0/1), does the LLM follow the *remaining* (Layer 3) rules better?

Setup: 10 tasks x 3 constraint counts (10, 15, 20) x 4 conditions, Claude Opus 4.6. For FORGE conditions, rules with deterministic Python checkers are classified as Layer 0/1 and removed from the prompt. The LLM only sees the "residual" rules that genuinely require reasoning.

Table 5a: Overall Compliance (all rules checked post-gen):

Condition	N=10	N=15	N=20
Baseline	0.870	0.887	0.880
FORGE Filter	0.570	0.633	0.625
Best-of-5	0.920	0.913	0.910

Condition	N=10	N=15	N=20
FORGE + BO5	0.620	0.727	0.650

Table 5b: Residual Compliance (Layer 3 rules only):

Condition	N=10	N=15	N=20
Baseline	0.889	0.815	0.871
FORGE Filter	**0.889**	**0.926**	**1.000**
Best-of-5	1.000	0.889	1.000
FORGE + BO5	**0.944**	**1.000**	**0.968**

Finding: Core thesis confirmed — residual compliance (Layer 3 rules the LLM actually sees) improves +11–13pp at N=15–20 when enforceable rules are removed. At N=20, the LLM follows all remaining rules perfectly. Overall compliance drops because filtered rules aren' t in the prompt, but in production these are enforced by Layer 0/1 mechanisms post-generation.

14.6 Experiment 6: SENTINEL — Zero-Rule Generation + Swarm Enforcement

Question: When the LLM sees **zero** rules and a swarm of sentinels enforces compliance externally, is compliance independent of rule count?

Setup: 10 tasks x 4 constraint counts (5, 10, 15, 20) x 2 conditions, Claude Opus 4.6. SENTINEL condition: zero-constraint prompt (just the task description, no constraints) + external sentinel swarm with 8 regex auto-fixers for Tier 1–2 items.

Compliance (pass rate):

Condition	N=5	N=10	N=15	N=20
Baseline	0.900	0.860	0.833	0.885
SENTINEL	0.700	0.700	0.693	0.750

Finding: SENTINEL is 1.2x more stable across rule counts (variation 0.057 vs 0.067) with constant O(1) prompt cost (57 tokens vs baseline's 84–244). Compliance is 70–75% with Tier 1–2 fixers only; full production deployment with 153+ sentinels achieves 94.3% (Section 6.10). Auto-fix resolves 3–18 items per condition (6–9% of checked rules).

14.7 Experiment 7: P13 Hyper-Isolated Verification

Question: Does one-constraint-per-LLM-call scoring outperform monolithic multi-constraint scoring?

Setup: 10 texts x 4 constraint counts (5–20). P13 Isolated (one constraint per LLM call) vs Monolithic (all constraints in one call), both Claude Opus 4.6, scored against deterministic ground truth.

N constraints	P13 Isolated (Opus 4.6)	Monolithic (Opus 4.6)	Delta
5	0.920	0.850	**+0.070**
10	0.875	0.825	**+0.050**
15	0.893	0.908	-0.015
20	0.913	0.888	**+0.025**

Finding: P13 isolated scoring shows +2.5 to +7pp advantage at 3 of 4 constraint counts (within noise at N=15). However, P13 requires a frontier-class model — smaller models produce unreliable results. Viable as Layer 4 enforcement at ∼$0.003/item (∼$0.30 per generation for N=100 items).

References

Allen Institute for AI. IFBench: Generalizing verifiable instruction following. In *NeurIPS Datasets and Benchmarks Track*, 2025. `https://github.com/allenai/IFBench`.

Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback, 2022.

Jian Chen et al. Coninstruct: Evaluating large language models on conflict detection and resolution in instructions, 2025a.

Jiao Chen et al. The instruction gap: When language models ignore their instructions, 2026a.

Liang Chen, Ruoyu Wang, and Ming Zhang. Draft-conditioned constrained decoding: Decoupling semantic planning from structural enforcement, 2026b.

Yuxuan Chen et al. Decoupling task-solving and output formatting in LLM generation, 2025b.

Council Mode Authors. Council mode: Mitigating hallucination and bias in LLMs via multi-agent consensus, 2026.

Guardrails AI. Guardrails: Validators and retry loops for production LLM outputs. Product documentation, 2025. `https://guardrailsai.com`.

Devansh Gupta, Shashi Narayan, and Pranav Joshi. Autoqual: An LLM agent for automated discovery of interpretable features for review quality assessment. In *EMNLP Industry Track*, 2025.

Vaibhav Gupta and Dinesh Sreenivasamurthy. Prose2policy: A practical LLM pipeline for translating natural-language access policies into executable rego, 2026.

Qichang Hu, Lei Zhang, and Wei Liu. Modex: Evaluator-free selection via semantic consensus across diverse LLM outputs, 2025.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Shiyang Li et al. On the effect of sampling diversity in scaling LLM inference, 2025.

Yulin Li, Tianyi Sun, and Xiaojun Huang. Coda: A context-decoupled hierarchical agent with reinforcement learning. In *WSDM*, 2026.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

Yang Liu et al. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *EMNLP*, 2023.

Sewon Min et al. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In *EMNLP*, 2023.

Ning Nie et al. Thinking before constraining: A unified decoding framework for large language models, 2026.

Jiwon Park, Hyunjun Kim, and Seungjin Lee. ATLAS-RTC: Closing the loop on LLM agent output with token-level runtime control, 2026.

Alberto Purpura, Bo Wang, Saurabh Badyal, Leo Beaufrand, and Luke Faulkner. Deconstructing instruction-following: A new benchmark for granular evaluation of large language model instruction compliance abilities. In *Proc. EACL*, 2026. arXiv:2601.18554.

Mathieu Ravaut, Shafiq Joty, and Nelson F. Liu. Context length alone hurts LLM performance despite perfect retrieval. In *Findings of EMNLP*, 2025.

Andrea Recchia, Samuel Spaulding, and Jonathan Pfeifer. Recursive language models for long-document understanding. Technical report, MIT CSAIL, 2025.

Zhen Ren et al. DAK-UCB: Diversity-aware prompt routing for LLMs and generative models, 2026.

Ankit Sharma, Rohit Gupta, and Neel Mehta. Hiflow: A hierarchical feedback-driven framework for constrained long-form text generation, 2026.

Lei Sun, Yanchen Huang, and Dong Chen. Robon: Robust best-of-n jailbreaking via reward model routing, 2025.

Yifan Tang, Kaiwei Tian, and Zhiyuan Li. Found in the middle: Calibrating positional attention bias in long-context LLMs. In *NeurIPS Workshop on Long-Context*, 2024.

Francisco Torres, Beatriz Martinez, and Carlos Alvarez. Metaglyph: Symbolic metalanguages for prompt compression, 2026.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Eric Wallace et al. The instruction hierarchy: Training LLMs to prioritize privileged instructions, 2024.

Yujie Wan et al. Fusion: Faithful unified synthesis over narratives for multi-source summarization. In *International Conference on Learning Representations (ICLR)*, 2026.

Qiang Wang et al. Autopatent: A multi-agent framework for automatic patent generation, 2024.

Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Zheng Wang, Tianqi Huang, and Junyu Li. Diverse sampling strategies for best-of-n language model alignment, 2025a.

Ziyan Wang, Li Chen, and Tong Zhang. Chain-of-thought can degrade instruction-following compliance in large language models, 2025b.

Yixiao Xie et al. Laser: Governing long-horizon agentic search via structured protocol and context register, 2025.

Jun Zhang et al. A multi-dimensional constraint framework for evaluating and improving instruction following in large language models, 2025.

Weihao Zhao, Kiran Patel, and Arjun Verma. Autorule: Reasoning chain-of-thought extracted rule-based rewards improve preference learning, 2025.

Jeffrey Zhou et al. Instruction-following evaluation for large language models, 2023.

Tianyang Zhu et al. Agenthallu: Benchmarking automated hallucination attribution of LLM-based agents, 2026.